eTracee：面向国产操作系统的轻量级 eBPF 攻击链可视化系统实施方案

1. 核心问题解决方案

1.1 攻击数据来源方案

为了验证eTracee系统的有效性并应对真实威胁，获取高质量的攻击数据是至关重要的。攻击数据不仅用于测试系统的检测能力，更是训练和优化AI异常检测模型的基础。本方案提出两种互补的数据获取策略：利用公开数据集进行模型训练与验证，以及在隔离环境中模拟攻击行为进行实时捕获。这两种方案相结合，既能保证数据的多样性和真实性，又能确保测试过程的安全可控。

1.1.1 方案一：利用公开数据集进行模型训练与验证

公开数据集是快速启动和验证AI模型的重要资源。它们通常包含了经过专家标注的、多样化的攻击样本，能够帮助我们快速建立基线模型，并评估其性能。

- **数据集选择：ADFA-LD**
  - **数据集介绍**：澳大利亚国防学院（Australian Defence Force Academy）发布的ADFA-LD（ADFA Linux Dataset）是一个专门用于Linux系统入侵检测研究的数据集。它包含了在Linux系统上收集的正常行为数据和多种类型的攻击数据，如Hydra-FTP暴力破解、Metasploit框架攻击、Webshell上传与执行等。该数据集的优势在于其攻击场景的多样性和真实性，能够很好地模拟真实世界中的攻击行为。
  - **数据格式**：ADFA-LD中的系统调用序列以数字编号的形式存储，每个数字代表一个特定的系统调用。为了将这些数字映射到具体的系统调用名称，需要获取并解析 `ADFA-LD+Syscall+List.txt` 文件。这个映射关系对于理解攻击行为的具体操作至关重要。
  - **应用方式**：ADFA-LD数据集将作为训练AI异常检测模型的主要基准数据。通过分析数据集中正常行为与攻击行为的系统调用序列模式，可以训练出能够区分正常与异常行为的模型。例如，可以提取系统调用的n-gram特征，或者构建系统调用频率的直方图，作为模型的输入。
- **数据集选择：新墨西哥大学数据集**
  - **数据集介绍**：新墨西哥大学（University of New Mexico）也发布了一系列用于系统调用异常检测研究的数据集。这些数据集同样包含了正常和异常的系统调用序列，并且在学术界被广泛使用，具有较高的权威性。
  - **应用方式**：新墨西哥大学的数据集可以作为ADFA-LD的补充，用于验证模型的泛化能力。通过在不同来源的数据集上进行测试，可以评估模型是否能够适应不同的系统环境

和攻击模式，从而避免模型过拟合于特定数据集。

1.1.2 方案二：在隔离环境中模拟攻击行为进行实时捕获

虽然公开数据集提供了丰富的攻击样本，但它们可能无法完全覆盖最新的攻击技术或特定于openEuler系统的攻击场景。因此，在隔离环境中模拟攻击行为并进行实时捕获，是获取最新、最相关攻击数据的关键。

- **环境搭建**
  - **容器化环境**：使用Docker或LXC等容器技术在openEuler 25.09上创建隔离的测试环境。容器具有轻量、快速启动和易于管理的特点，非常适合进行大规模的攻击模拟实验。可以在容器中部署各种应用服务，如Web服务器、数据库等，作为攻击目标。
  - **虚拟机环境**：在虚拟机（如KVM、VirtualBox）中部署openEuler 25.09，可以提供一个与宿主机完全隔离的环境。虚拟机环境更接近于真实的物理服务器，可以模拟更复杂的网络拓扑和攻击场景。
- **攻击模拟方法**
  - **利用已知漏洞**：针对已知的容器逃逸漏洞（如CVE-2019-5736、CVE-2022-0185）和内核漏洞（如Dirty Cow）进行模拟攻击。这些漏洞的利用代码通常可以在GitHub等平台上找到。通过复现这些攻击，可以捕获到与权限提升、容器逃逸等高危行为相关的系统调用序列。
  - **使用攻击工具**：运行Metasploit、Hydra、Nmap等渗透测试工具，模拟真实的攻击行为。例如，使用Hydra对SSH服务进行暴力破解，使用Metasploit对Web应用进行漏洞利用。这些工具会产生大量的网络连接和系统调用，为eTracee系统提供丰富的测试数据。
  - **执行恶意脚本**：编写并运行模拟恶意行为的脚本，如异常文件访问（如尝试读取 `/etc/shadow` ）、权限提升尝试（如执行 `sudo` 命令）、反向Shell连接等。这些脚本可以模拟攻击者在获取初步访问权限后的横向移动和持久化操作。
- **数据捕获**
  - **eBPF程序部署**：在隔离环境中部署eTracee的eBPF捕获模块，实时记录攻击行为产生的系统调用序列。eBPF程序可以设置为捕获所有系统调用，或者根据预设的规则进行过滤，以减少数据量。
  - **数据存储**：将捕获的数据以JSON Lines（ `.jsonl` ）格式存储。这种格式每行一个JSON对象，便于后续的数据处理和分析。每条记录应包含时间戳、进程ID、进程名、系统调用号、系统调用参数等关键信息。

1.2 AI模型优化方案

考虑到项目开发周期紧迫，AI模型的选择需要在开发成本（难度和时间）与实际效果之间做出平衡。优先选择利用现有成熟技术和模型，同时规划长期自定义模型的开发路径。

1.2.1 优先方案：基于无监督学习的轻量级模型

无监督学习模型无需标注数据，可以充分利用收集到的大量正常行为数据进行训练，非常适合在攻击数据稀缺的初期阶段使用。

- **模型选择：Autoencoder（自编码器）**
  - **原理**：自编码器是一种神经网络，由编码器和解码器两部分组成。编码器将输入数据压缩成一个低维的潜在表示（latent representation），解码器则尝试从这个潜在表示中重构原始输入。在训练过程中，模型会学习如何有效地表示正常数据。当输入异常数据时，由于其模式与正常数据差异较大，重构过程会产生较高的误差（reconstruction error）。这个重构误差可以作为异常分数，误差越大，异常程度越高。这种方法非常适合系统调用异常检测，因为我们通常拥有大量的正常行为数据，而攻击样本相对稀少且难以穷举。
  - **实现**：可以使用PyTorch或TensorFlow等深度学习框架构建一个简单的自编码器模型。输入层的大小可以根据系统调用的种类数量来确定，隐藏层的维度可以远小于输入层，以实现数据的压缩。
  - **优势**：自编码器模型结构简单，训练速度快，模型轻量，非常适合部署在资源受限的环境中进行实时检测。此外，由于它只使用正常数据进行训练，因此能够检测到未知的、新型的攻击。
- **特征工程**
  - **系统调用频率直方图**：在eBPF程序中，可以统计每个进程在固定时间窗口（如100毫秒）内各种系统调用的调用次数，形成一个频率直方图。这个直方图可以看作是一个向量，向量的每个维度代表一种系统调用，其值代表该时间窗口内该调用的次数。这个向量可以作为自编码器的输入。
  - **数据传输**：将计算好的直方图数据通过eBPF的ring buffer发送到用户态。ring buffer是一种高效的、无锁的通信机制，非常适合在内核和用户空间之间传递大量数据。
- **模型训练与部署**
  - **训练**：使用收集到的正常系统调用数据（如通过方案一和方案二获取的正常行为数据）来训练自编码器模型。训练的目标是最小化重构误差。

Generated by Kimi AI

- **部署**：将训练好的模型集成到eTracee的后端服务中。当实时捕获到新的系统调用数据时，将其转换为频率直方图，输入到模型中进行预测。模型会输出一个重构误差，作为该行为的异常评分。

1.2.2 备选方案：基于传统机器学习算法的模型

如果自编码器模型的效果不佳，或者希望尝试更简单的实现，可以考虑使用传统的机器学习算法。

- **模型选择：Isolation Forest（孤立森林）**
  - **原理**：孤立森林是一种基于决策树的异常检测算法。它通过随机选择特征和分割值来构建多棵决策树。异常点由于与正常点距离较远，更容易被孤立（即更早地被分到叶子节点）。通过计算样本在所有树上的平均路径长度，可以判断其异常程度。
  - **实现**：scikit-learn库提供了高效的Isolation Forest实现，可以直接调用。
  - **优势**：实现简单，计算效率高，对高维数据有较好的效果，并且不需要大量的训练数据。
- **模型选择：One-Class SVM**
  - **原理**：One-Class SVM是一种支持向量机（SVM）的变体，它只使用一类样本（通常是正常样本）进行训练。其目标是在特征空间中找到一个超平面，将正常数据与原点最大化地分离。当新的数据点被映射到该空间时，如果它位于超平面的另一侧（即靠近原点），则被判定为异常。
  - **实现**：scikit-learn库同样提供了One-Class SVM的实现。
  - **优势**：对小样本数据有较好的效果，并且模型的决策边界比较清晰，具有一定的可解释性。

1.2.3 自定义模型方案（长期规划）

在项目后期，如果希望进一步提升检测的准确率和召回率，可以考虑构建更复杂的自定义模型。

- **模型设计：基于序列的深度学习模型**
  - **模型选择**：可以考虑使用LSTM（长短期记忆网络）或GRU（门控循环单元）等循环神经网络（RNN）。这些模型能够捕捉系统调用序列中的时间依赖关系，对于检测基于序列的攻击（如代码注入、ROP/JOP攻击）非常有效。

Generated by Kimi AI

- **数据预处理**：需要将系统调用序列转换为词嵌入（word embedding）向量，作为模型的输入。每个系统调用可以被看作一个“词”，其对应的嵌入向量可以通过训练学习得到。

- **训练策略**
  - **数据增强**：通过对正常数据进行扰动（如随机删除、替换或插入系统调用），可以生成更多的训练样本，提高模型的鲁棒性。
  - **对抗训练**：引入对抗样本（即经过微小扰动的恶意样本），可以迫使模型学习到更鲁棒的特征，提高其对对抗性攻击的抵抗力。

- **挑战与风险**
  - **开发周期**：自定义模型的开发、训练和调优需要较长的时间，可能超出项目的14周周期。
  - **计算资源**：深度学习模型对计算资源（CPU/GPU）要求较高，可能会影响系统的实时性。需要在模型复杂度和系统性能之间做出权衡。

2. 项目计划评价与优化建议

2.1 潜在设计冗余分析

2.1.1 规则引擎与AI模型的功能重叠

- **问题**：规则引擎和AI模型都可以用于检测异常行为，这可能导致功能上的重叠。例如，一个简单的规则（如“禁止非root用户执行 `sudo` ”）和一个AI模型（通过学习正常用户的 `sudo` 使用模式来检测异常）可能在检测同一类攻击时产生重叠。

- **优化建议**：明确划分规则引擎和AI模型的职责。规则引擎应专注于检测已知的、明确的、模式固定的攻击行为，例如特定的系统调用序列、敏感文件访问、网络连接等。它的优势在于低延迟、高确定性，可以快速响应已知的威胁。而AI模型则应专注于检测未知的、复杂的、模式不固定的异常行为，例如通过分析系统调用的频率、序列、参数等统计特征来发现潜在的攻击。AI模型的优势在于其泛化能力和对新型攻击的发现能力。通过这样的职责划分，可以形成一个互补的检测体系：规则引擎作为第一道防线，快速拦截已知的恶意行为；AI模型作为第二道防线，对绕过规则引擎的未知攻击进行深度分析和检测。

2.1.2 数据存储与处理的冗余

- **问题**：事件数据在eBPF程序、用户态接收器、规则引擎、AI模型等多个模块之间传递，可能存在数据格式转换和存储的冗余。例如，eBPF程序可能以一种紧凑的格式发送数据，而

Generated by Kimi AI

用户态接收器为了便于处理，可能会将其转换为JSON格式，规则引擎和AI模型又可能需要将其转换为其他格式。这种多次转换不仅增加了系统的复杂性，也可能带来性能开销。

- **优化建议**：设计一个统一、高效的数据格式，并在整个系统中保持一致。推荐使用JSON Lines（`.jsonl`）格式，因为它既具有人类可读性，又便于程序解析，并且支持流式处理。在后端服务中，可以设计一个高效的数据流处理框架，负责数据的接收、解析、分发和存储。此外，可以考虑引入消息队列（如Redis）来解耦各个模块。eBPF程序将数据发送到消息队列，规则引擎、AI模型等模块作为消费者从队列中获取数据进行处理。这种架构可以提高系统的可扩展性和容错性，避免单个模块的性能瓶颈影响整个系统。

2.2 优化实现步骤建议

2.2.1 并行开发与模块化设计

- **建议**：将项目分解为多个独立的、松耦合的模块，如eBPF捕获模块、规则引擎模块、AI模型模块、前端可视化模块等。每个模块都有明确的接口和职责，可以由不同的团队成员并行开发。
- **优势**：并行开发可以显著缩短项目的整体开发周期。模块化设计使得每个模块都可以独立进行测试和优化，提高了代码的质量和可维护性。当需要添加新功能或修改现有功能时，只需要改动相应的模块，而不会影响到整个系统。

2.2.2 敏捷开发与迭代优化

- **建议**：采用敏捷开发模式，将整个14周的项目周期划分为多个短小的迭代周期（例如，每两周一个迭代）。在每个迭代开始时，确定本次迭代需要完成的功能列表（Backlog），并在迭代结束时交付一个可用的、经过测试的软件版本。
- **优势**：敏捷开发强调快速交付和持续反馈。通过频繁的迭代，可以及时发现和解决问题，快速响应需求的变化。每个迭代都交付一个可用的版本，可以让用户尽早地看到系统的进展，并提供反馈，从而确保最终交付的产品能够满足用户的实际需求。

2.2.3 性能优化与安全加固

- **建议**：在项目早期就考虑性能优化和安全加固，而不是等到项目后期再进行。例如，在编写eBPF程序时，就要注意其对系统性能的影响，避免使用过于复杂的逻辑和大量的内存。在设计后端API时，就要考虑权限控制、输入验证、防止SQL注入等安全问题。
- **优势**：在项目早期就进行性能优化和安全加固，可以避免在项目后期进行大规模的重构，从而节省时间和成本。一个高性能、高安全性的系统，更容易获得用户的信任和认可。此外，将性能和安全作为设计的一部分，可以培养团队成员的性能意识和安全意识，提高整个团队的技术水平。

Generated by Kimi AI

3. 系统架构与技术选型

3.1 系统架构

3.1.1 数据采集层

- **eBPF程序（C）**：负责在内核空间捕获系统调用事件，进行初步的过滤和聚合，并将数据发送到用户态。

3.1.2 数据处理与存储层

- **用户态接收器（Go）**：负责从eBPF程序接收数据，进行格式转换和持久化存储。
- **数据库（SQLite）**：用于存储事件数据、规则配置、图谱数据等。
- **消息队列（Redis）**：用于解耦各个模块，实现异步数据处理。

3.1.3 业务逻辑层

- **规则引擎（Go）**：负责根据预定义的规则对事件流进行匹配，触发告警。
- **AI模型（Python）**：负责对事件流进行异常检测，输出异常评分。
- **图谱构建服务（Go/Python）**：负责将事件流转换为关系图谱，并提供图谱查询接口。

3.1.4 可视化与交互层

- **前端（HTML/JS/D3.js）**：负责展示实时事件流、攻击链图谱、告警信息等，并提供交互功能。

3.2 技术选型

3.2.1 eBPF开发

- **libbpf**：用于开发eBPF程序，支持CO-RE（Compile Once – Run Everywhere）模式，提高程序的可移植性。
- **bpftrace**：用于快速验证eBPF程序的功能，编写简单的脚本进行系统调用捕获。

3.2.2 后端开发

- **Go**：用于开发用户态接收器、规则引擎、图谱构建服务等后端服务，具有高性能、高并发的特点。
- **Python**：用于开发AI模型、报告生成等模块，拥有丰富的机器学习库和数据处理工具。

Generated by Kimi AI

3.2.3 前端开发

- **D3.js**：用于实现攻击链图谱的可视化，提供丰富的图形布局和交互功能。
- **WebSocket**：用于实现前后端的实时数据通信。

3.2.4 数据库

- **SQLite**：轻量级的嵌入式数据库，适合存储事件数据、规则配置等。
- **Redis**：高性能的内存数据库，用于实现消息队列和缓存。

4. 14周开发计划

4.1 第一阶段：环境准备与eBPF基础（第1–4周）

- **第1周**：在openEuler 25.09上搭建开发环境，运行eBPF示例程序。
- **第2周**：使用libbpf编写最简eBPF程序，捕获系统调用事件并导出数据。
- **第3周**：实现系统调用过滤与聚合功能。
- **第4周**：实现轻量级规则引擎，支持基本的规则匹配和告警。

4.2 第二阶段：数据存储与后端架构（第5–8周）

- **第5周**：设计并实现事件持久化方案，使用SQLite存储事件数据。
- **第6周**：构建后端服务骨架，实现数据流与WebSocket接口。
- **第7周**：设计关系图谱模型，实现图谱构建与查询API。
- **第8周**：实现前端可视化PoC，展示攻击链图谱。

4.3 第三阶段：AI模型与可视化增强（第9–11周）

- **第9周**：增强前端可视化功能，支持图谱过滤、时间回放等。
- **第10周**：设计并实现轻量级AI异常检测模型。
- **第11周**：实现AI模型与规则引擎的联动，升级告警体系。

4.4 第四阶段：报告生成与最终交付（第12–14周）

- **第12周**：实现自动化报告生成功能，支持PDF/HTML格式。
- **第13周**：进行整合测试、性能优化与安全加固。

Generated by Kimi AI

- **第14周**：完成最终交付，包括源码、部署脚本、用户手册、演示视频等。